

ELLIPSOID METHOD AND THE AMAZING ORACLES*

WAI-SHING LUK†

Abstract. The ellipsoid method is a powerful optimization technique that offers distinct advantages over interior-point methods, as it does not require the evaluation of all constraint functions. This makes it the optimal choice for convex problems with numerous or even infinite constraints. The method employs an ellipsoid as a search space and relies on a separation oracle to provide cutting planes for updating it. It is worth noting that the significance of the separation oracle is often overlooked. This article evaluates the utility of the ellipsoid method in three distinct applications: robust convex optimization, semidefinite programming, and parametric network optimization. The effectiveness of separation oracles is assessed for each application. Furthermore, this article addresses the implementation issues associated with the ellipsoid method, including the utilization of parallel cuts for updating the ellipsoid. In certain cases, the use of parallel cuts has been observed to reduce computation time, as evidenced in the context of FIR filter design. The article also considers discrete optimization, demonstrating how the ellipsoid method can be applied to problems involving quantized discrete design variables. The additional effort in oracle implementation is limited to locating the nearest discrete solutions.

1. Introduction. The reputation of the ellipsoid method is negatively impacted by its perceived slower performance in solving large-scale convex problems when compared to the interior-point method. This perception is, however, an unfair one. In contrast to the interior-point method, the ellipsoid method does not require the explicit evaluation of all constraint functions. By contrast, the method employs an ellipsoid as a search space, requiring only a separation oracle that furnishes a *cutting plane* (§ 2). This method is particularly well-suited to problems that involve a moderate number of design variables but have a large number of constraints, or even an infinite number of constraints. Some have criticized the method, claiming that it is unable to leverage sparsity. Nevertheless, while the ellipsoid method is unable to leverage sparsity, the separation oracle is capable of exploiting specific structural types.

Despite decades of research into the ellipsoid method [1], the importance of the separation oracle is often overlooked. This article examines three specific applications: robust convex optimization, network optimization, and semidefinite programming. The effectiveness of separation oracles is assessed for each application.

Robust optimization incorporates parameter uncertainties into the optimization problem by analyzing the worst-case scenario. The objective is to find a solution that is both reliable and performs optimally under a range of possible parameter values within a specified set of uncertainties. A robust counterpart of a convex problem preserves its convexity, despite the number of constraints growing to infinity. This renders the ellipsoid method an excellent choice for addressing such problems. For further details, see § 3.1.

Furthermore, an illustration is provided of a network optimization scenario in which the ellipsoid method can be utilized. The separation oracle involves the construction of a cutting plane by finding a negative cycle within a network graph. There are algorithms available for finding negative cycles that leverage network locality and other

*This work was supported by ...

†Fudan University

properties, resulting in an efficient implementation of oracles. For a more detailed discussion, please refer to § 3.2.

Meanwhile, § 3.3 addresses concerns related to matrix inequalities. Recall that the use of Cholesky or LDLT decomposition allows for the efficient checking of the positive definiteness of a symmetric matrix. If a symmetric matrix A , with dimensions of $m \times m$, encounters a non-positive diagonal entry during decomposition, and the process is terminated at row p , then A cannot be positive definite. In such cases, a witness vector v can be constructed to certify that A is not positive definite. The row-based decomposition and lazy evaluation technique enable the cutting plane to be constructed in $O(p^3)$, thus allowing its use in efficient oracle implementations.

The implementation of the ellipsoid method is discussed in greater detail in § 4. In essence, the method generates a sequence of ellipsoids whose volume is uniformly decreased at each step, thereby ensuring a systematic and consistent approach to volume reduction. The ellipsoid is typically represented as follows:

$$\{x \mid (x - x_c)P^{-1}(x - x_c) \leq 1\},$$

where $x_c \in \mathbb{R}^n$ is the center of the ellipsoid. The matrix $P \in \mathbb{R}^{n \times n}$ is a symmetric positive definite matrix of order n . In each iteration, the ellipsoid method performs updates to both x_c and P . Although updating ellipsoids is a relatively straightforward process that has been implemented for decades, we show that the cost can be reduced by an additional n^2 floating point operations by splitting the matrix P into two parts, namely, κ and Q . This yields in the following form:

$$\{x \mid (x - x_c)Q^{-1}(x - x_c) \leq \kappa\}.$$

Moverover, § 4.3 addresses the utilization of parallel cuts. Some researchers have suggested that this technique does not result in significant improvements. Nevertheless, our findings indicate that in scenarios where specific constraints are subject to narrow upper and lower bounds, such as in the context of FIR filter designs, the incorporation of parallel cuts can markedly reduce the runtime. Furthermore, we demonstrate that when the ellipsoid method is implemented with precision, any update, whether it employs a single cut or a parallel cut, necessitates at most one square root operation.

In many practical engineering problems, some design variables may be constrained to discrete forms. Since the cutting plane method requires only a separation oracle, it can also be used for discrete problems. The only additional effort in the oracle implementation is the finding of the closest discrete solutions.

2. Cutting plane Method Revisited.

2.1. Convex Feasibility Problem. Let $\mathcal{K}$ be a compact and convex subset of $\mathbb{R}^n$. Consider the following feasibility problem:

1. Find a point $x^* \in \mathcal{K}$ in $\mathcal{K}$, or
2. Determine if $\mathcal{K}$ is empty, i.e. if it has no feasible solution.

A separation oracle, also known as a cutting plane oracle, is a method used to delineate a convex set that serves as an input to a cutting plane method. When a separation oracle, denoted by Ω , is queried at a given point $x_0 \in \mathbb{R}^n$, it can produce one of the following outputs:

1. It asserts that x_0 belongs to $\mathcal{K}$, or
2. Returns a hyperplane separating the point x_0 from the set $\mathcal{K}$:

$$g^\top(x - x_0) + \beta \leq 0, \beta \geq 0, g \neq 0, \forall x \in \mathcal{K}.$$

The pair of (g, β) is called a *cutting plane* because it eliminates the half-space defined by the equation $\{x \mid g^\top(x - x_0) + \beta > 0\}$ from the search space. The following observations are made:

- If $\beta = 0$, indicating that x_0 is on the boundary of the half-space, the cutting plane is called a *central-cut*.
- If $\beta > 0$, indicating that x_0 is inside the half-space, the cutting plane is called a *deep-cut*.
- If $\beta < 0$, indicating that x_0 is outside the half-space, the cutting plane is called a *shadow-cut*.

The convex set $\mathcal{K}$ is typically defined by a set of inequalities $f_j(x) \leq 0$ or $f_j(x) < 0$ for $j = 1 \cdots m$, where $f_j(x)$ represents a convex function. The vector $g \equiv \partial f(x_0)$ is defined as the *sub-gradient* of a convex function f at the point x_0 if $f(z) \geq f(x_0) + g^\top(z - x_0)$. Thus, the cut (g, β) can be expressed as $(\partial f(x_0), f(x_0))$. Note that if $f(x)$ is differentiable, then we can simply take $\partial f(x_0) = \nabla f(x_0)$.

The cutting plane method consists of two main elements: a separation oracle, denoted by Ω , and a search space, denoted by $\mathcal{S}$, which is initially set to a sufficiently large to encompass $\mathcal{K}$. For example,

- Polyhedron $\mathcal{P} = \{z \mid Cz \preceq d\}$.
- Ellipsoid $\mathcal{E} = \{z \mid (z - x_c)^\top P^{-1}(z - x_c) \leq 1\}$.
- Interval $\mathcal{I} = [l, u]$ (for one-dimensional problem).

Let us denote the center of the current set, denoted by $\mathcal{S}$, as x_c . The following is a basic outline of the methodology underlying the cutting plane method:

1. **Initialization:** The initial stage of the method involves defining a search space $\mathcal{S}$ that is guaranteed to contain a point x^* .
2. **Iteration:** In each iteration, the separation oracle is queried at the center x_c . If x_c is in $\mathcal{K}$, then the iteration is terminated.
3. **Update:** The smaller search space, denoted by $\mathcal{S}^+$, is computed and contains the half-space from step 2.
4. **Repeat:** Repeat steps 2 and 3 until $\mathcal{S}$ is either empty or sufficiently small.

2.2. From Feasibility to Optimization. Let us now turn our attention to the following consideration:

$$\begin{aligned} & \text{minimize} && f_0(x), \\ & \text{subject to} && x \in \mathcal{K}. \end{aligned}$$

In this case, the objective function, $f_0(x)$, can be either a convex function or a quasi-convex function. The aforementioned optimization problem is treated as a feasibility problem with an additional constraint, namely that $f_0(x) \leq \gamma$, where $\gamma \in \mathbb{R}$ is called the best-so-far value of $f_0(x)$. Accordingly, the problem can be reformulated as follows:

$$\begin{aligned} & \text{minimize} && \gamma, \\ & \text{subject to} && \Phi(x, \gamma) \leq 0, \\ & && x \in \mathcal{K}, \end{aligned}$$

where $\Phi(x, \gamma) \leq 0$ is the γ -sublevel set of $f_0(x)$ when $f_0(x)$ is quasi-convex. For every x , $\Phi(x, \gamma)$ is a non-increasing function of γ , i.e., $\Phi(x, \gamma') \leq \Phi(x, \gamma)$ whenever $\gamma' \geq \gamma$. Denote $\mathcal{K}_\gamma$ as the new constraint set.

One straightforward approach to solving the optimization problem is to perform a binary search on γ and solve the corresponding feasibility problems at each value of γ . An alternative approach is to update the current best estimate of γ whenever a feasible solution x_0 is found such that $\Phi(x_0, \gamma) = 0$.

The following is a basic outline of the operational procedure of the cutting plane method (optim):

1. **Initialization:** The initial stage of the process entails defining a search space $\mathcal{S}$ that is guaranteed to contain a solution, x^* .
2. **Iteration:** In each iteration, the separation oracle is queried at the point x_c . A subgradient of the function at x_c must then be computed. This results in the generation of a half-space that is guaranteed to contain x^* .
3. If $x_c \in \mathcal{K}_\gamma$, update γ such that $\Phi(x_c, \gamma) = 0$.
4. **Update:** The smaller $\mathcal{S}^+$ that contains the half-space from step 2 is computed.
5. **Repeat:** Repeat steps 2 to 4 until $\mathcal{S}$ is either empty or sufficiently small.

Generic cutting plane method (Optim)

- **Given** an initial $\mathcal{S}$ known to contain $\mathcal{K}_\gamma$.
- **Repeat**
 1. Select a point x_0 in $\mathcal{S}$
 2. Query the separation oracle at x_0
 3. **If** $x_0 \in \mathcal{K}_\gamma$, update γ so that $\Phi(x_0, \gamma) = 0$.
 4. Update $\mathcal{S}$ to a smaller set that covers:

$$\mathcal{S}^+ = \mathcal{S} \cap \{z \mid g^\top(z - x_0) + \beta \leq 0\}$$

5. **If** $\mathcal{S}^+ = \emptyset$ or it is small enough, exit.

We assume that the oracle takes responsibility for this.

```
def cutting_plane_optim(omega, space, gamma, options=Options()):
    x_best = None
    for niter in range(options.max_iters):
        cut, t1 = omega.assess_optim(space.xc(), t)
        if t1 is not None: # better gamma obtained
            gamma = t1
            x_best = copy.copy(space.xc())
```

```

        status = space.update_central_cut(cut)
    else:
        status = space.update_deep_cut(cut)
    if status != CutStatus.Success or space.tsq() < options.tol:
        return x_best, gamma, niter
return x_best, gamma, options.max_iters

```

2.3. Example: Profit Maximization. This example is taken from the article “Robust Optimization for Profit Maximization” by Aliabadi (2013) [2]. We will consider the following *short-run* profit maximization problem:

$$\begin{aligned}
 (2.1) \quad & \text{maximize} && p(Ax_1^\alpha x_2^\beta) - v_1 x_1 - v_2 x_2, \\
 & \text{subject to} && x_1 \leq k, \\
 & && x_1 > 0, x_2 > 0,
 \end{aligned}$$

where the variable A represents the scale of production, while the variables α and β denote output elasticities. The x_i and v_i terms refer to the quantity and price of the i th input, respectively. The term $Ax_1^\alpha x_2^\beta$ is the Cobb-Douglas production function, which is a widely accepted model used to represent the relationship between inputs and outputs in production. The quantity of x_1 is constrained by the constant k . Please be advised that the aforementioned formulation is not in convex form. To begin, we will reformulate the problem as follows:

$$\begin{aligned}
 & \text{maximize} && \gamma, \\
 & \text{subject to} && \gamma + v_1 x_1 + v_2 x_2 \leq pAx_1^\alpha x_2^\beta, \\
 & && x_1 \leq k, \\
 & && x_1 > 0, x_2 > 0.
 \end{aligned}$$

By means of a change of variables, the following convex form of Eq. 2.1 can be obtained:

$$\begin{aligned}
 & \text{maximize} && \gamma, \\
 & \text{subject to} && \log(\gamma + v_1 e^{y_1} + v_2 e^{y_2}) - (\alpha y_1 + \beta y_2) \leq \log(pA), \\
 & && y_1 \leq \log k,
 \end{aligned}$$

{#eq:profit-in-cvx-form} where $y_1 = \log x_1$ and $y_2 = \log x_2$.

```

class ProfitOracle(OracleOptim):
    def __init__(self, params, elasticities, price_out):
        unit_price, scale, limit = params
        self.log_pA = math.log(unit_price * scale)
        self.log_k = math.log(limit)
        self.price_out = price_out
        self.elasticities = elasticities

    def assess_optim(self, y, t):
        if (fj := y[0] - self.log_k) > 0.0: # constraint
            grad = np.array([1.0, 0.0])

```

```

    return (grad, fj), None

log_Cobb = self.log_pA + self.elasticities.dot(y)
q = self.price_out * np.exp(y)
vx = q[0] + q[1]
if (fj := math.log(t + vx) - log_Cobb) >= 0.0:
    grad = q / (t + vx) - self.elasticities
    return (grad, fj), None

gamma = np.exp(log_Cobb) - vx
grad = q / (t + vx) - self.elasticities
return (grad, 0.0), t

```

Some readers may recognize that the problem can also be written in a geometric program by introducing one additional variable [2].

3. Amazing Oracles.

- Robust convex optimization
 - oracle technique: affine arithmetic
- Parametric network potential problem
 - oracle technique: negative cycle detection
- Semidefinite programming
 - oracle technique: Cholesky decomposition

3.1. Robust Convex Optimization. In essence, robust optimization addresses parameter uncertainties by formulating problems that consider worst-case scenarios. This approach ensures more reliable and robust solutions when dealing with uncertainty. This study addresses the issue of profit maximization using a robust geometric programming approach that accounts for interval uncertainty. The authors examine the well-established Cobb-Douglas production function and propose an approximate equivalent of the robust counterpart, utilizing piecewise convex linear approximations. This approximation is expressed in the form of a geometric programming problem. To illustrate the impact of uncertainty, we will present an illustrative example.

In the context of mathematical modeling, interval uncertainties refer to the representation of model parameters as intervals. In this study, the authors examine how interval uncertainties affect the model parameters. The article presents upper and lower piecewise convex linear approximations of the robust counterpart, which can be solved using interior point methods.

For the purposes of this discussion, we will consider:

$$(3.1) \quad \begin{array}{ll} \text{minimize} & \sup_{q \in \mathcal{Q}} f_0(x, q), \\ \text{subject to} & f_j(x, q) \leq 0, \forall q \in \mathcal{Q}, j = 1, 2, \dots, m, \end{array}$$

where q represents a set of varying parameters. The issue can be rephrased as follows:

$$\begin{array}{ll} \text{minimize} & \gamma, \\ \text{subject to} & f_0(x, q) \leq \gamma, \\ & f_j(x, q) \leq 0, \forall q \in \mathcal{Q}, j = 1, 2, \dots, m. \end{array}$$

3.1.1. Algorithm. The oracle is responsible for determining the following:

- If $f_j(x_0, q) > 0$ for some j and $q = q_0$, then the cut (g, β) is equal to $(\partial f_j(x_0, q_0), f_j(x_0, q_0))$.
- If $f_0(x_0, q) \geq \gamma$ for some $q = q_0$, then the cut (g, β) is equal to $(\partial f_0(x_0, q_0), f_0(x_0, q_0) - \gamma)$.
- Otherwise, x_0 is feasible, then
 - Let $q_{\max} = \arg \max_{q \in \mathcal{Q}} f_0(x_0, q)$.
 - $\gamma := f_0(x_0, q_{\max})$.
 - The cut (g, β) is equal to $(\partial f_0(x_0, q_{\max}), 0)$.

3.1.2. Example: Robust Profit Maximization. Let us revisit the profit maximization problem in § 2.3. The model parameters are subject to uncertainty over a given interval. Let us now consider the case in which the parameters $\alpha, \beta, p, v_1, v_2$, and k are subject to interval uncertainties, as outlined in [2]:

$$\begin{array}{rclcl}
 \alpha - \varepsilon_1 & \leq & \hat{\alpha} & \leq & \alpha + \varepsilon_1 \\
 \beta - \varepsilon_2 & \leq & \hat{\beta} & \leq & \beta + \varepsilon_2 \\
 p - \varepsilon_3 & \leq & \hat{p} & \leq & p + \varepsilon_3 \\
 v_1 - \varepsilon_4 & \leq & \hat{v}_1 & \leq & v_1 + \varepsilon_4 \\
 v_2 - \varepsilon_5 & \leq & \hat{v}_2 & \leq & v_2 + \varepsilon_5 \\
 k - \varepsilon_6 & \leq & \hat{k} & \leq & k + \varepsilon_6
 \end{array}$$

The problem formulation of the robust counterpart considering the worst-case scenario is:

$$\begin{array}{ll}
 \max & \gamma \\
 \text{s.t.} & \log(\gamma + \hat{v}_1 e^{y_1} + \hat{v}_2 e^{y_2}) - (\hat{\alpha} y_1 + \hat{\beta} y_2) \leq \log(\hat{p} A) \\
 & y_1 \leq \log \hat{k}.
 \end{array}$$

In [2], the authors propose the use of piecewise convex linear approximations as a proximate approximation of the robust counterpart, thereby facilitating greater solvability through the use of interior-point algorithms. This approach requires the development of a significant amount of programming, which may be challenging for some teams. However, the resulting solutions are inherently imprecise, which is something to consider. However, this can be readily addressed through the cutting plane method. It should be noted that in this simple example, the worst-case scenario occurs when:

- $\hat{p} = p - e_3, k = \bar{k} - e_3$
- $v_1 = \bar{v}_1 + e_3, v_2 = \bar{v}_2 + e_3$,
- if $y_1 > 0, \alpha = \bar{\alpha} - e_1$, else $\alpha = \bar{\alpha} + e_1$
- if $y_2 > 0, \beta = \bar{\beta} - e_2$, else $\beta = \bar{\beta} + e_2$

It is even possible to reuse the original oracle to compose the robust counterpart.

```

class ProfitRbOracle(OracleOptim):
    def __init__(self, params, elasticities, price_out, vparams):
        e1, e2, e3, e4, e5 = vparams

```

```

self.elasticities = elasticities
self.e = [e1, e2]
unit_price, scale, limit = params
params_rb = unit_price - e3, scale, limit - e4
self.omega = ProfitOracle(
    params_rb, elasticities, price_out + np.array([e5, e5])
)

def assess_optim(self, y, t):
    a_rb = copy.copy(self.elasticities)
    for i in [0, 1]:
        a_rb[i] += -self.e[i] if y[i] > 0.0 else self.e[i]
    self.omega.elasticities = a_rb
    return self.omega.assess_optim(y, t)

```

It should be noted that the “argmax” may be non-convex, which may make it challenging to solve. For more complex problems, one potential approach is to utilize affine arithmetic as a computational aid [3].

3.2. Multi-parameter Network Problems. In the context of network theory, a directed graph, denoted by $G = (V, E)$, represents a network. Let us consider the following:

$$\begin{aligned}
 &\text{minimize} && \gamma, \\
 &\text{subject to} && u_i - u_j \leq h_{ij}(x, \gamma), \forall (i, j) \in E, \\
 &\text{variables} && x, u,
 \end{aligned}$$

where $h_{ij}(x, \gamma)$ is the weight function of edge (i, j) .

It is assumed that the network is of a considerable size, but that the number of parameters is relatively limited. The problem has a feasible solution if and only if G contains no negative cycles. Let $\mathcal{C}$ be a set of all cycles of G . The problem can be formulated as follows:

$$\begin{aligned}
 &\text{minimize} && \gamma, \\
 &\text{subject to} && W_k(x, \gamma) \geq 0, \forall C_k \in \mathcal{C}, \\
 &\text{variables} && x,
 \end{aligned}$$

where C_k is a cycle of G :

$$W_k(x, \gamma) = \sum_{(i,j) \in C_k} h_{ij}(x, \gamma).$$

The minimum cycle ratio (MCR) problem is a fundamental problem in the analysis of directed graphs. Given a directed graph, the MCR problem seeks to find the cycle with the minimum ratio of the sum of the edge weights to the number of edges in the cycle. In other words, the MCR problem seeks to find the “tightest” cycle in the graph, where the tightness of a cycle is measured by the ratio of the total weight of the cycle to its length.

The MCR problem has numerous applications in the analysis of discrete event systems, including digital circuits and communication networks. It is closely related to other problems in graph theory, such as the shortest path problem and the maximum flow problem. Consequently, efficient algorithms for solving the MCR problem are of great practical importance.

3.2.1. Negative Cycle Detection Algorithm. The most time-consuming part of the proposed method is the negative cycle detection, which underscores the importance of selecting an appropriate negative cycle detection algorithm. There are numerous methods for detecting negative cycles in weighted graphs [4]. Tarjan’s algorithm [5] is one of the fastest in practice and is widely regarded as a benchmark for this purpose [4,6].

Howard’s method is a minimum cycle ratio (MCR) algorithm that employs a policy iteration algorithm to find the minimum cycle ratio of a directed graph. The algorithm maintains a set of candidate cycles and proceeds by iteratively updating the cycle with the minimum ratio until convergence is reached.

The separation oracle is only required to determine:

- If a negative cycle C_k exists under x_0 , then the cut $(g, \beta) = (-\partial W_k(x_0), -W_k(x_0))$ ■
- If $f_0(x_0) \geq \gamma$, then the cut $(g, \beta) = (\partial f_0(x_0), f_0(x_0) - \gamma)$.
- Otherwise, x_0 is feasible, then
 - $\gamma := f_0(x_0)$.
 - The cut $(g, \beta) = (\partial f_0(x_0), 0)$

3.2.2. Example: Optimal matrix scalings under the min-max-ratio criterion. The following example is taken from [7]. As stated by [7], optimal matrix scaling has a number of practical applications. One such application is in the field of linear programming, where groups of constraints and groups of variables may represent the same physical commodity for which common measurement units are employed. Another area of application is in telecommunications, where matrix scaling can be employed to optimize the transmission of signals. Furthermore, matrix scaling has been employed in approximation theory to approximate functions of multiple variables by the sum of functions of fewer variables. Furthermore, matrix scaling has been employed in Gaussian elimination, a prevalent method for solving systems of linear equations, with the objective of enhancing the numerical stability of the algorithm.

Let us consider a matrix $A \in \mathbb{R}^{N \times N}$. A *symmetric scaling* of A is defined as a matrix B of the form UAU^{-1} , where U is a nonnegative diagonal matrix of the same dimension. In accordance with the *min-max criterion*, the objective is to minimize the largest absolute value of B ’s elements [7, (Program 3)]:

$$\begin{aligned} & \text{minimize} && \pi \\ & \text{subject to} && 1 \leq u_i |a_{ij}| u_j^{-1} \leq \Pi, \forall a_{ij} \neq 0, \\ & && \pi, u_1 \cdot u_N \text{ positive.} \end{aligned}$$

The authors demonstrate that the problem of determining the optimal symmetric scalings under the min-max criterion can be transformed into a single-parameter network optimization problem. This can be solved efficiently using parameteric network algorithms.

Another possible criterion is to minimize the ratio of the largest absolute value of the element B to the smallest. One rationale for employing this criterion is that high ratios impede the efficacy of the simplex method. With this *min-max-ratio* criterion, the symmetric scaling problem can be formulated as [7, (Program 8)]:

$$\begin{aligned} & \text{minimize} && \pi/\psi \\ & \text{subject to} && \psi \leq u_i |a_{ij}| u_j^{-1} \leq \Pi, \forall a_{ij} \neq 0, \\ & && \pi, \psi, u_1 \cdot u_N \text{ positive.} \end{aligned}$$

Let k' denotes $\log(|k|)$. By taking the logarithm of the variables, the aforementioned programming can be transformed into a two-parameter network problem:

$$\begin{aligned} & \text{minimize} && \pi' - \psi' \\ & \text{subject to} && u'_i - u'_j \leq \pi' - a'_{ij}, \forall a_{ij} \neq 0, \\ & && u'_j - u'_i \leq a'_{ij} - \psi', \forall a_{ij} \neq 0, \\ & \text{variables} && \pi', \psi', u'. \end{aligned}$$

where $x = (\pi', \psi')^\top$. The authors of [7] assert that they have developed an algorithm for solving multi-parameter problems. Nevertheless, we were unable to identify any follow-up publications that corroborate this assertion. It is noteworthy that the cutting plane method readily extends the single-parameter network algorithm to accommodate multi-parameter problems.

In this application, the function $h_{ij}(x)$ is defined as follows:

$$h_{ij}(x) = \begin{cases} -\pi' + a'_{ij}, & \forall a_{ij} \neq 0, \\ \psi' - a'_{ji}, & \forall a_{ji} \neq 0, \end{cases}$$

Fast algorithms for finding a negative cycle can be found in [8,9]. Further applications to clock skew scheduling can be found in [10].

3.3. Problems Involving Matrix Inequalities. Consider the following problem:

$$\begin{aligned} & \text{find} && x, \\ & \text{subject to} && F(x) \succeq 0, \end{aligned}$$

where $F(x)$ is a matrix-valued function, $A \succeq 0$ denotes a positive semidefinite matrix. It should be recalled that a matrix A is positive semidefinite if and only if $v^\top A v \geq 0$ for all $v \in \mathbb{R}^N$. The problem can be transformed into the following form:

$$\begin{aligned} & \text{find} && x, \\ & \text{subject to} && v^\top F(x) v \geq 0, \forall v \in \mathbb{R}^N. \end{aligned}$$

Consider the case where $v^T F(x)v$ is concave for all $v \in \mathbb{R}^N$ with respect to x , then the above problem is a convex programming. The problem can be reduced to *semidefinite programming* if the function $F(x)$ is linear with respect to x , i.e., $F(x) = F_0 + x_1 F_1 + \dots + x_n F_n$, where $F_0, F_1, F_2, \dots, F_n$ are constants.

In the field of convex optimization, a **linear matrix inequality (LMI)** is defined as an expression of the form:

$$A(y) = A_0 + y_1 A_1 + y_2 A_2 + \dots + y_m A_m \succeq 0,$$

where $y = [y_i, i = 1, \dots, n]$ is a real vector, $A_0, A_1, A_2, \dots, A_n$ are symmetric matrices, and $A(y) \succeq 0$ is a generalized inequality, meaning that $A(y)$ is a positive semidefinite matrix.

This linear matrix inequality defines a convex constraint on the variable y . There are efficient numerical methods for determining the feasibility of an LMI (e.g., whether there exists a vector y such that $A(y) \succeq 0$), as well as for solving convex optimization problems with LMI constraints.

3.3.1. Cholesky decomposition algorithm. The Cholesky decomposition algorithm is a method used in linear algebra to decompose a Hermitian, positive-definite matrix into the product of a lower triangular matrix and its conjugate transpose. This decomposition offers an efficient numerical solution, including Monte Carlo simulations.

The Cholesky decomposition of a Hermitian positive-definite matrix A is a unique decomposition where $A = LL^*$, with L being a lower triangular matrix containing real and positive diagonal entries, and L^* representing the conjugate transpose of L . It is possible to express every real-valued symmetric positive-definite matrix and Hermitian positive-definite matrix as a Cholesky decomposition.

If A is a real matrix that is symmetric and positive-definite, it can be decomposed as $A = LL^T$. Here, L represents a real lower triangular matrix with positive diagonal entries.

The Cholesky and LDLT decompositions are matrix decomposition methods utilized in linear algebra for disparate purposes, exhibiting distinctive properties.

The Cholesky decomposition is a method for decomposing a Hermitian, positive-definite matrix into the product of a lower triangular matrix and its conjugate transpose. The Cholesky decomposition is typically more rapid and numerically stable method than the LDLT decomposition. Nevertheless, the input matrix must be positive definite for this to be effective.

The LDLT decomposition, a variant of the LU decomposition that applies to positive-definite symmetric matrices, is a more versatile option as it can be applied to a wider range of matrices and does not require them to be positive-definite. The LDLT decomposition factors a matrix into the product of a lower triangular matrix, a diagonal matrix, and the transpose of the lower triangular matrix. This decomposition is as fast as the Cholesky decomposition, but does not require any square roots, making it faster and more numerically stable.

$$\begin{aligned}\mathbf{A} = \mathbf{LDL}^\top &= \begin{pmatrix} 1 & 0 & 0 \\ L_{21} & 1 & 0 \\ L_{31} & L_{32} & 1 \end{pmatrix} \begin{pmatrix} D_1 & 0 & 0 \\ 0 & D_2 & 0 \\ 0 & 0 & D_3 \end{pmatrix} \begin{pmatrix} 1 & L_{21} & L_{31} \\ 0 & 1 & L_{32} \\ 0 & 0 & 1 \end{pmatrix} \\ &= \begin{pmatrix} D_1 & & \\ L_{21}D_1 & L_{21}^2D_1 + D_2 & \\ L_{31}D_1 & L_{31}L_{21}D_1 + L_{32}D_2 & L_{31}^2D_1 + L_{32}^2D_2 + D_3 \end{pmatrix}. \end{aligned}$$

If A is real, the following recursive relations apply for the entries of D and L :

$$D_j = A_{jj} - \sum_{k=1}^{j-1} L_{jk} L_{jk}^* D_k,$$

$$L_{ij} = \frac{1}{D_j} \left(A_{ij} - \sum_{k=1}^{j-1} L_{ik} L_{jk}^* D_k \right) \quad \text{for } i > j.$$

Once more, the pattern of access enables the entire computation to be performed in-place.

The Cholesky or LDLT decomposition can be computed using either row-based or column-based methods:

- Column-Based: In this approach, the computation is conducted in a column-wise manner. The inner loops calculate the current column using a matrix-vector product that accumulates the effects of previous columns.
- Row-Based: In this approach, the calculations are conducted row by row. The inner loops are responsible for computing the current row, which is achieved by solving a triangular system involving previous rows.

The selection of each outer loop index results in a unique Cholesky algorithm, named after the portion of the matrix updated by the fundamental operation within the inner loops. The choice between a row-based or column-based method depends on the specific requirements of the problem, as well as the system properties, such as memory layout and access patterns. The row-based decomposition and lazy evaluation technique allow for the cutting plane construction to be completed with exactness in $O(p^3)$. This allows for the effective implementation of oracles.

The following is the algorithm written in Python:

```
def factor(self, getA):
    T = self.T
    for i in range(self.n): # from 0 to n-1
        for j in range(i+1): # from 0 to i
            d = getA(i, j) - np.dot(T[:j, i], T[j, :j])
            T[i, j] = d
            if i != j:
                T[j, i] = d / T[j, j]
```

```

    if d <= 0.: # strictly positive
        self.p = i
        return
    self.p = self.n

```

The Cholesky decomposition provides a witness vector that certifies a matrix is not positive definite. If a matrix fails the Cholesky decomposition, it is not positive definite. During the decomposition process, the diagonal of the lower diagonal matrix should be calculated by finding the square root of a value, denoted as x . If x is less than zero, this indicates that the matrix is not positive definite. This failure serves as evidence that the matrix in question is not positive definite.

In the event that the Cholesky decomposition is unsuccessful due to a negative diagonal element, this indicates that the leading principal submatrix up to that point is not positive definite. One of the standard basis vectors is used as the confirming vector. This basis vector includes a 1 in the position corresponding to the failed diagonal element and zeros in all other positions. When this vector is pre-multiplied by the original matrix and then post-multiplied by its transpose, the result will be a negative value, which serves as a witnessing vector.

The following is the algorithm, written in Python:

```

def witness(self):
    p = self.p
    n = p + 1
    v = np.zeros(n)
    v[p] = 1
    for i in range(p, 0, -1): # backward substitution
        v[i-1] = -np.dot(self.T[i-1, i:n], v[i:n])
    return v, -self.T[p, p]

```

The oracle should perform a *row-based* Cholesky decomposition such that $F(x_0) = R^T R$. The notation $A_{:p,:p}$ is used to denote a submatrix $A(1:p, 1:p) \in \mathbb{R}^{p \times p}$. If the Cholesky decomposition fails at row p , there exists a vector e_p , defined as $(0, 0, \dots, 0, 1)^T \in \mathbb{R}^p$. This can be expressed as follows:

- $v = R_{:p,:p}^{-1} e_p$, and
- $v^T F_{:p,:p}(x_0) v < 0$.

The cut (g, β) is then given by the following equation:

$$(-v^T \partial F_{:p,:p}(x_0) v, -v^T F_{:p,:p}(x_0) v).$$

3.3.2. Example: Matrix Norm Minimization. Let $A(x) = A_0 + x_1 A_1 + \dots + x_n A_n$. Problem $\min_x \|A(x)\|$ can be reformulated as

$$\begin{aligned} & \text{minimize} && \gamma, \\ & \text{subject to} && \begin{pmatrix} \gamma I_m & A(x) \\ A^T(x) & \gamma I_n \end{pmatrix} \succeq 0. \end{aligned}$$

A binary search on γ can be used for this problem.

3.3.3. Example: Estimation of Correlation Function.

3.4. Random Field [11]. *Random field*, also known as *stochastic process*, can be regarded as an indexed family of random variables denoted as $\{Z(\mathbf{s}) : \mathbf{s} \in D\}$, where D is a subset of d -dimensional Euclidean space $\mathbb{R}^d$. To specify a stochastic process, the joint probability distribution function of any finite subset $(Z(\mathbf{s}_1), \dots, Z(\mathbf{s}_n))$ must be given in a consistent way, which is called *distribution* of the process. For ease of analysis, a random field is often assumed to be with *Gaussian* distribution and is called Gaussian random field.

A random field has several key properties useful in practical problems. The field is *stationary* under translations, or *homogeneous*, if the distribution is unchanged when the point set is translated. The field is *isotropic* if the distribution is invariant under any rotation of the whole points in the parameter space. We study the homogeneous isotropic field in this paper.

The *covariance* C and *correlation* R of a stochastic process are defined by:

$$C(\mathbf{s}_i, \mathbf{s}_j) = \text{cov}(Z(\mathbf{s}_i), Z(\mathbf{s}_j)) = E[(Z(\mathbf{s}_i) - E[Z(\mathbf{s}_i)])(Z(\mathbf{s}_j) - E[Z(\mathbf{s}_j)])]$$

and

$$R(\mathbf{s}_i, \mathbf{s}_j) = C(\mathbf{s}_i, \mathbf{s}_j) / \sqrt{C(\mathbf{s}_i, \mathbf{s}_i)C(\mathbf{s}_j, \mathbf{s}_j)}$$

respectively for all $\mathbf{s}_i, \mathbf{s}_j \in D$, where $E[Z(\mathbf{s})]$ denotes the expectation of $Z(\mathbf{s})$. Thus a process is homogeneous if C and R depend only on the separation vector $\mathbf{h} = \mathbf{s}_i - \mathbf{s}_j$. Furthermore, it is isotropic if C and R depend upon $\mathbf{h}$ only through its length h , i.e.,

$$C(\mathbf{s}_i, \mathbf{s}_j) = C(\mathbf{h}) = C(h),$$

$$(3.2) \quad R(\mathbf{s}_i, \mathbf{s}_j) = R(\mathbf{h}) = R(h) = C(h)/C(0).$$

If we denote $C(0)$, the variance of $Z(\mathbf{s})$, as σ^2 , then the relationship between covariance and correlation is $C(h) = \sigma^2 R(h)$.

When the two components are considered, the measurement data can still be regarded as a Gaussian random field, but the correlation function will have a discontinuity at the origin. We call this phenomenon “nugget effect” [12].

$$\begin{aligned} \min_{\kappa, p} \quad & \|\Omega(p) + \kappa I - Y\| \\ \text{s.t.} \quad & \Omega(p) \succcurlyeq 0, \kappa \geq 0. \end{aligned}$$

Let $\rho(h) = \sum_i^n p_i \Psi_i(h)$, where p_i 's are the unknown coefficients to be fitted Ψ_i 's are a family of basis functions. The covariance matrix $\Omega(p)$ can be recast as:

$$\Omega(p) = p_1 F_1 + \dots + p_n F_n,$$

where $\{F_k\}_{i,j} = \Psi_k(\|s_j - s_i\|_2)$.

4. Ellipsoid Method Revisited. Some History of the Ellipsoid Method [1]. Introduced by Shor and Yudin and Nemirovskii in 1976. It used to show that linear programming (LP) is polynomial-time solvable (Kachiyan 1979), settled the long-standing problem of determining the theoretical complexity of LP. In practice, however, the simplex method runs much faster than the method, although its worst-case complexity is exponential.

4.1. Basic Ellipsoid Method. An ellipsoid $\mathcal{E}_k(x_k, P_k)$ is specified as a set

$$\{x \mid (x - x_k)P_k^{-1}(x - x_k) \leq 1\},$$

where $x_k \in \mathbb{R}^n$ is the center of the ellipsoid and $P_k \in \mathbb{R}^{n \times n}$ is a positive definite matrix.

Example: For a 2D ellipsoid centered at (0,0) with $P = \begin{bmatrix} 4 & 0 \\ 0 & 1 \end{bmatrix}$, the set would be all points (x, y) satisfying $x^2/4 + y^2 \leq 1$.

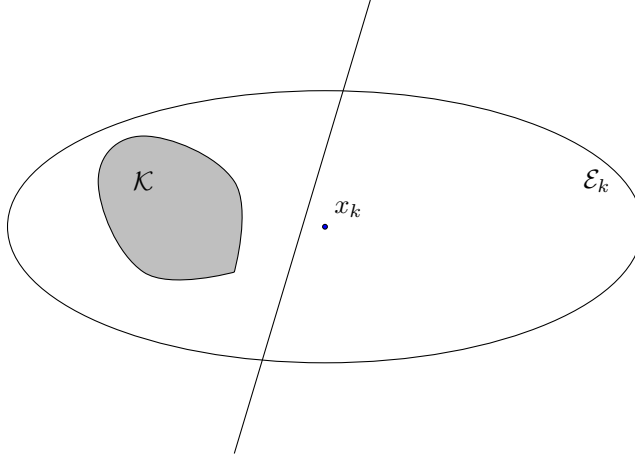


Figure 4.1: Ellipsoid, feasible region, and cut

Updating the ellipsoid (deep-cut)

Calculation of minimum volume ellipsoid covering:

$$\mathcal{E}_k \cap \{z \mid g^\top(z - x_k) + \beta \leq 0\}$$

Let $\tilde{g} = P_k g$, $\tau^2 = g^\top P_k g$. We can make the following observations:

1. If $n \cdot \beta < -\tau$ (shallow cut), then no smaller ellipsoid can be found.
2. If $\beta > \tau$, then intersection is empty.
3. Otherwise,

$$x_c^+ = x_c - \frac{\rho}{\tau^2} \tilde{g}, \quad P^+ = \delta \cdot \left(P - \frac{\sigma}{\tau^2} \tilde{g} \tilde{g}^\top \right)$$

where

$$\rho = \frac{\tau + nh}{n + 1}, \quad \sigma = \frac{2\rho}{\tau + \beta}, \quad \delta = \frac{n^2(\tau^2 - \beta^2)}{(n^2 - 1)\tau^2}$$

Example: For $n = 2$, $\tau = 2$, $\beta = 1$: $\rho = (2 + 2 * 1)/3 = 1.33$, $\sigma = 2 * 1.33/3 = 0.89$, $\delta = 4 * (4 - 1)/(3 * 4) = 1$

Even better, split P into two variables $\kappa \cdot Q$. Let $\tilde{g} = Q \cdot g$, $\omega = g^\top \tilde{g}$, $\tau = \sqrt{\kappa \cdot \omega}$.

$$x_c^+ = x_c - \frac{\rho}{\omega} \tilde{g}, \quad Q^+ = Q - \frac{\sigma}{\omega} \tilde{g} \tilde{g}^\top, \quad \kappa^+ = \delta \cdot \kappa$$

Reduce n^2 multiplications per iteration. Note that:

- The determinant of Q decreases monotonically.
- The range of δ is $(0, \frac{n^2}{n^2-1})$

4.2. Central Cut. A Special case of when $\beta = 0$. Deserve a separate implement because it is much simpler. Let $\tilde{g} = Q g$, $\tau = \sqrt{\kappa \cdot \omega}$,

$$\rho = \frac{\tau}{n+1}, \quad \sigma = \frac{2}{n+1}, \quad \delta = \frac{n^2}{n^2-1}.$$

Example: For $n = 3$, $\tau = 2$: $\rho = 2/4 = 0.5$, $\sigma = 2/4 = 0.5$, $\delta = 9/8 = 1.125$

4.3. Parallel Cuts. Oracle returns a pair of cuts instead of just one. The pair of cuts is given by g and (β_1, β_2) such that:

$$\begin{aligned} g^\top(x - x_c) + \beta_1 &\leq 0, \\ g^\top(x - x_c) + \beta_2 &\geq 0, \end{aligned}$$

for all $x \in \mathcal{K}$.

Only linear inequality constraint can produce such parallel cut:

$$l \leq a^\top x + b \leq u, \quad L \preceq F(x) \preceq U.$$

Usually, provide faster convergence.

Updating the ellipsoid.

Let $\tilde{g} = Q g$, $\tau^2 = \kappa \cdot \omega$.

- If $\beta_1 > \beta_2$, intersection is empty.
- If $\beta_1 \beta_2 < -\tau^2/n$, no smaller ellipsoid can be found.
- If $\beta_2^2 > \tau^2$, it reduces to deep-cut with $\alpha = \alpha_1$.

Otherwise,

$$x_c^+ = x_c - \frac{\rho}{\omega} \tilde{g}, \quad Q^+ = Q - \frac{\sigma}{\omega} \tilde{g} \tilde{g}^\top, \quad \kappa^+ = \delta \kappa.$$

where

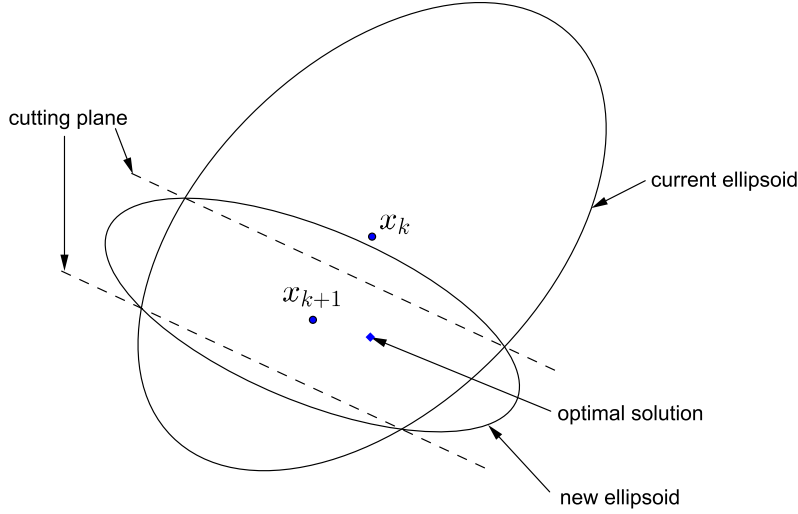


Figure 4.2: Parallel cuts

$$\begin{aligned}
\bar{\beta} &= (\beta_1 + \beta_2)/2 \\
\xi^2 &= (\tau^2 - \beta_1^2)(\tau^2 - \beta_2^2) + (n(\beta_2 - \beta_1)\bar{\beta})^2, \\
\sigma &= (n + (\tau^2 - \beta_1\beta_2 - \xi)/(2\bar{\beta}^2))/(n + 1), \\
\rho &= \bar{\beta} \cdot \sigma, \\
\delta &= (n^2/(n^2 - 1))(\tau^2 - (\beta_1^2 + \beta_2^2)/2 + \xi/n)/\tau^2.
\end{aligned}$$

Example: For $n = 2$, $\tau = 2$, $\beta_1 = -1$, $\beta_2 = 1$: $\bar{\beta} = 0$, $\xi^2 = (4 - 1)(4 - 1) + 0 = 9 \Rightarrow \xi = 3$ $\sigma = (2 + (4 - (-1) - 3)/0 \Rightarrow$ undefined (special case handled separately) This shows the need for special handling when $\bar{\beta} = 0$.

4.3.1. Example: FIR filter design. A typical structure of digital Finite Impulse Response (FIR) filter is shown in Fig. 4.3, where the coefficients $h[0], h[1], \dots, h[n-1]$ must be determined to meet given specifications. Usually, they can be manually designed using windowing or frequency-sampling techniques [13].

However, the experience and knowledge of designers are highly demanded in this kind of design methods. Moreover, there is no guarantee about the design's quality. Therefore, the optimization-based techniques (e.g. [14], more reference) have attracted tons of research effort. In this kind of method, facilitated with growing computing resources and efficient optimization algorithms, the solution space can be effectively explored.

In optimization algorithms, what is particularly interesting is the convex optimization. If a problem is in a convex form, it can be efficiently and optimally solved. Convex optimization techniques are also implementable in designing FIR filters, including the Parks-McClellan algorithm [16], METEOR [17], and peak-constrained least-squares (PCLS) [18,19]. In the mentioned articles, with the help of exchange algorithms (e.g. Remez exchange algorithm), certain FIR filter design problems can be formed as linear or quadratic programs. They are two simple forms of convex optimization

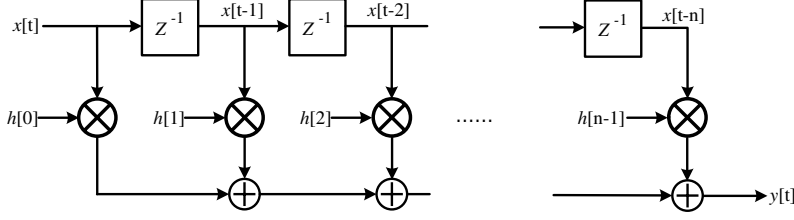


Figure 4.3: A typical structure of an FIR filter [15].

problems, which can be optimally solved with existing algorithms, such as the interior-point method [20]. Tempted by the optimality, more efforts were devoted to forming the problem convex. Particularly, in [14], via spectral decomposition [21], the problem of designing an FIR filter with magnitude constraints on frequency-domain is formulated as a convex optimization problem. More examples are provided in [22].

Its time response is

$$(4.1) \quad y[t] = \sum_{k=0}^{n-1} h[k]u[t-k]$$

where $\mathbf{h} = (h(0), h(1), \dots, h(n-1))$ is the filter coefficients. Its frequency response $H : [0, \pi] \rightarrow \mathbb{C}$ is

$$(4.2) \quad H(\omega) = \sum_{m=0}^{n-1} h(m)e^{-jm\omega}$$

where $j = \sqrt{-1}$, n is the order of the filter. The design of a filter with magnitude constraints is often formulated as a constraint optimization problem as the form

$$\begin{aligned} \min & \gamma \\ \text{s.t.} & f(\mathbf{x}) \leq \gamma \\ & g(\mathbf{x}) \leq 0. \end{aligned}$$

where $\mathbf{x}$ is the vector of design variables, $g(\mathbf{x})$ represents the characteristics of the desirable filter and $f(\mathbf{x})$ is the performance metric to be optimized. For example, the magnitude constraints on frequency domain are expressed as

$$(4.3) \quad L(\omega) \leq |H(\omega)| \leq U(\omega), \forall \omega \in (-\infty, +\infty)$$

where $L(\omega)$ and $U(\omega)$ are the lower and upper (nonnegative) bounds at frequency ω respectively. Note that $H(\omega)$ is 2π periodic and $H(\omega) = \overline{H(-\omega)}$. Therefore, we can only consider the magnitude constraint on $[0, \pi]$ [14].

Generally, the problem might be difficult to solve, since we can only obtain the global optimal solution with resource-consuming methods, such as branch-and-bound [22]. However, the situation is totally different if the problem is convex, where $f(\mathbf{x})$ and $g(\mathbf{x})$ are convex functions. In such a case, the problem can be optimally solved with many efficient algorithms.

Attracted by the benefits, the authors of [14] transformed (?), originally non-convex, into a convex form via spectral decomposition:

$$(4.4) \quad L^2(\omega) \leq R(\omega) \leq U^2(\omega), \forall \omega \in (0, \pi)$$

where $R(\omega) = \sum_{i=-n+1}^{n-1} r(t)e^{-j\omega t} = |H(\omega)|^2$ and $\mathbf{r} = (r(-n+1), r(-n+2), \dots, r(n-1))$ are the autocorrelation coefficients. Especially, $\mathbf{r}$ can be determined by $\mathbf{h}$, with the following equation vice versa [14]:

$$(4.5) \quad r(t) = \sum_{i=-n+1}^{n-1} h(i)h(i+t), t \in \mathbb{Z}.$$

where $h(t) = 0$ for $t < 0$ or $t > n - 1$.

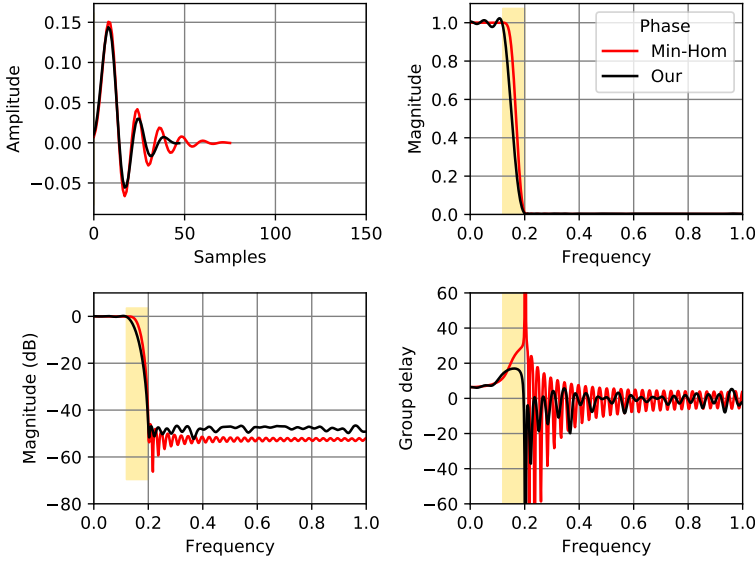


Figure 4.4: Result

4.3.2. Example: Maximum Likelihood estimation. Consider

$$\begin{aligned} \min_{\kappa, p} \quad & \log \det(\Omega(p) + \kappa \cdot I) + \text{Tr}((\Omega(p) + \kappa \cdot I)^{-1}Y), \\ \text{s.t.} \quad & \Omega(p) \succeq 0, \kappa \geq 0. \end{aligned}$$

Note that the first term is concave, the second term is convex. However, if there are enough samples such that Y is a positive definite matrix, then the function is convex within $[0, 2Y]$. Therefore, the following problem is convex:

$$\begin{aligned}
\min_{\kappa, p} \quad & \log \det V(p) + \text{Tr}(V(p)^{-1}Y), \\
\text{s.t.} \quad & \Omega(p) + \kappa \cdot I = V(p) \\
& 0 \preceq V(p) \preceq 2Y, \kappa > 0.
\end{aligned}$$

4.4. Discrete Optimization. Many engineering problems can be formulated through convex/geometric programming, such as digital circuit sizing. However, in ASIC design, there is frequently a limited number of cell types to select from in the cell library. This means that some design variables are discrete. We can map the design variables to integers to represent the discrete version as Mixed-Integer Convex programming (MICP).

What are the issues with existing methods? They are primarily based on relaxation. The more relaxed solution is used as the lower bound, and then the branch-and-bound method is applied to find the discrete optimal solution. It should be noted that the branch-and-bound method does not utilize the convexity of the issue. What if only constraints regarding discrete data could be evaluated?

Typically, a more relaxed optimal solution (convex) is obtained beforehand. After that, the optimized discrete solution is obtained using an exhaustive neighborhood search. However, tight constraints can cause a significant difference between the discrete and relaxed continuous optimal solutions. Enumerating the discrete domains can be challenging.

Consider:

$$\begin{aligned}
& \text{minimize} \quad f_0(x), \\
& \text{subject to} \quad f_j(x) \leq 0, \forall j = 1, 2, \dots, \\
& \quad \quad \quad x \in \mathbb{D},
\end{aligned}$$

where $f_0(x)$ and $f_j(x)$ are “convex”. Note that some design variables are discrete. The oracle looks for a nearby discrete solution x_d of x_c with the cutting plane:

$$g^T(x - x_d) + \beta \leq 0, \beta \geq 0, g \neq 0.$$

Note that the cut may be a shallow cut. Suggestion: use as many different cuts as possible for each iteration (e.g. round-robin the evaluation of constraints).

4.4.1. Example: Multiplierless FIR Filter Design. However, there are still many filter design problems that are non-convex, such as multiplierless FIR filter design problems. Note that in Fig. 4.3, each coefficient associated with a multiplier unit makes the filter power-hungry, especially in *application specific integrated circuits* (ASIC). Fortunately, if each coefficient is quantized and represented as a sum of Signed Power-of-Two (SPT), a multiplierless filter can be implemented. Such coefficients can be uniquely represented by a Canonical Signed-Digit (CSD) code with a minimum number of non-zero digits[23]. In this case, it confines the multiplication to addition and shift operations. The coefficient $0.40625 = 13/32$ can be written as $2^{-1} - 2^{-3} + 2^{-5}$. Thus, the multiplier can be replaced with three shifters and two adders at a much lower cost. However, the coefficient quantization constraint is non-convex, making the convex

optimization algorithm not directly applicable. A similar case is the consideration of the finite word-length effect [24].

Attracted by the benefits of this “multiplier-free” approach, many efforts have been devoted to its design techniques. For its general problems, integer programming (e.g. [24–27]) can be implemented to achieve the optimal solution. However, it requires excessive computational resources. Other heuristic techniques, such as genetic algorithm [28] and dynamic-programming-like method [29], also have inefficiency. If the quantization constraint is the only non-convex constraint in the design problem, a lower bound can be efficiently obtained by solving the relaxed problem [22]. Then to make the solution feasible, it can be rounded to the nearest CSD code or used as a starting point of a local search algorithm to obtain a better solution [30]. However, neither method guarantees the feasibility of the final solution. Besides, the local search problem remains non-convex. Therefore, the adopted algorithm may also be inefficient, such as branch-and-bound in [30].

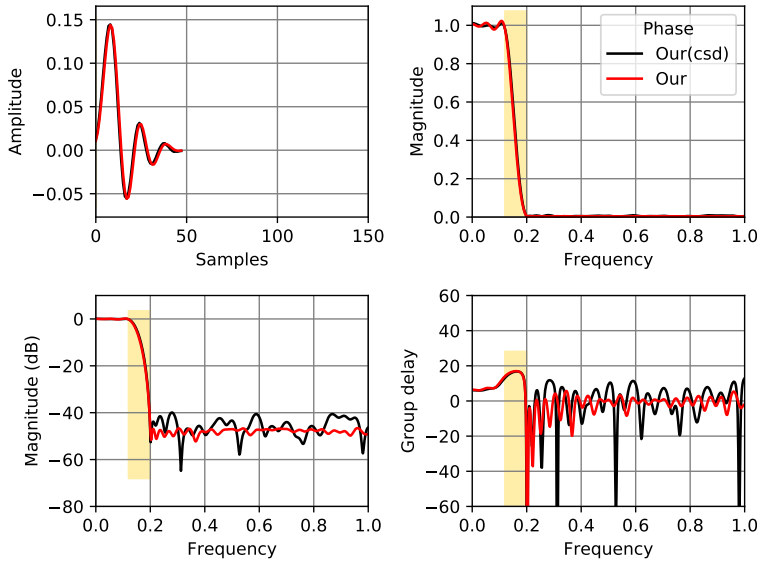


Figure 4.5: Result

5. Concluding Remarks. While the ellipsoid method may be perceived as slower than interior-point methods for solving convex problems, it offers distinct advantages, such as the ability to handle problems with a large or infinite number of constraints. Techniques like parallel cuts and efficient implementations have helped to improve the performance of the ellipsoid method, making it a valuable tool in the optimization landscape. Finally, rather than viewing the ellipsoid method as a competitor to other optimization techniques, it should be seen as a companion, with each method offering unique strengths that can be leveraged to solve a wide range of optimization problems effectively.

References.

- [1] R.G. Bland, D. Goldfarb, M.J. Todd, The ellipsoid method: A survey, *Operations Research* 29 (1981) 1039–1091.
- [2] H. Aliabadi, M. Salahi, Robust geometric programming approach to profit maximization with interval uncertainty, *Computer Science Journal of Moldova* 21 (2013) 86–96.
- [3] X. Liu, W.-S. Luk, Y. Song, P. Tang, X. Zeng, Robust analog circuit sizing using ellipsoid method and affine arithmetic, in: *Proceedings of the 2007 Asia and South Pacific Design Automation Conference*, IEEE Computer Society, 2007: pp. 203–208.
- [4] B.V. Cherkassky, A.V. Goldberg, Negative-cycle detection algorithms, *Mathematical Programming* 85 (1999) 277–311.
- [5] R.E. Tarjan, *Shortest paths*, AT&T Bell Laboratories, 1981.
- [6] A. Dasdan, Experimental analysis of the fastest optimum cycle ratio and mean algorithms, *ACM Transactions on Design Automation of Electronic Systems* 9 (2004) 385–418.
- [7] J.B. Orlin, U.G. Rothblum, Computing optimal scalings by parametric network algorithms, *Mathematical Programming* 32 (1985) 1–10.
- [8] A. Dasdan, R.K. Gupta, Faster maximum and minimum mean cycle algorithms for system-performance analysis, *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 17 (1998) 889–899.
- [9] A. Dasdan, Experimental analysis of the fastest optimum cycle ratio and mean algorithms, *ACM Transactions on Design Automation of Electronic Systems (TODAES)* 9 (2004) 385–418.
- [10] X. Zhou, W.-S. Luk, H. Zhou, F. Yang, C. Yan, X. Zeng, Multi-parameter clock skew scheduling, *Integration, the VLSI Journal* 48 (2015) 129–137.
- [11] O. Schabenberger, C.A. Gotway, *Statistical Methods for Spatial Data Analysis*, Chapman & Hall/CRC, Florida, 2005.
- [12] P.J. Diggle, P.J.R. Jr., *Model-based Geostatistics*, Springer, New York, 2007.
- [13] A.V. Oppenheim, R.W. Schaffer, J.R. Buck, others, *Discrete-time signal processing*, Prentice hall Englewood Cliffs, NJ, 1989.
- [14] S.-P. Wu, S. Boyd, L. Vandenberghe, FIR filter design via spectral factorization and convex optimization, in: *Applied and Computational Control, Signals, and Circuits*, Springer, 1999: pp. 215–245.
- [15] S.K. Mitra, Y. Kuo, *Digital signal processing: A computer-based approach*, McGraw-Hill New York, 2006.
- [16] T.W. Parks, J.H. McClellan, Chebyshev approximation for nonrecursive digital filters with linear phase, *Circuit Theory, IEEE Transactions On CT-19* (1972) 189–194.
- [17] K. Steiglitz, T.W. Parks, J.F. Kaiser, METEOR: A constraint-based FIR filter design program, *Signal Processing, IEEE Transactions On* 40 (1992) 1901–1909.
- [18] I.W. Selesnick, M. Lang, C.S. Burrus, Constrained least square design for FIR filters without specified transition bands, *Signal Processing, IEEE Transactions On* 44 (1996) 1879–1892.
- [19] J.W. Adams, J.L. Sullivan, Peak-constrained least-squares optimization, *Signal Processing, IEEE Transactions On* 46 (1998) 306–321.
- [20] S. Boyd, L. Vandenberghe, *Convex optimization*, Cambridge university press, 2009.

- [21] T.N.T. Goodman, C.A. Micchelli, G. Rodriguez, and S. Seatzu, Spectral factorization of laurent polynomials, *Advances Comput. Math.* 7 (1997) 429–454.
- [22] T.N. Davidson, Enriching the art of FIR filter design via convex optimization, *Signal Processing Magazine, IEEE* 27 (2010) 89–101.
- [23] G.W. Reitwiesner, Binary Arithmetic, *Advances in Computers* 1 (1960) 231–308.
- [24] Y.C. Lim, S. Parker, A. Constantinides, Finite word length FIR filter design using integer programming over a discrete coefficient space, *Acoustics, Speech and Signal Processing, IEEE Transactions On* 30 (1982) 661–664.
- [25] D.M. Kodek, Design of optimal finite wordlength FIR digital filters using integer programming techniques, *Acoustics, Speech and Signal Processing, IEEE Transactions On* 28 (1980) 304–308.
- [26] Y. Lim, S. Parker, FIR filter design over a discrete powers-of-two coefficient space, *Acoustics, Speech and Signal Processing, IEEE Transactions On* 31 (1983) 583–591.
- [27] Y.C. Lim, R. Yang, D. Li, J. Song, Signed power-of-two term allocation scheme for the design of digital filters, *Circuits and Systems II: Analog and Digital Signal Processing, IEEE Transactions On* 46 (1999) 577–584.
- [28] D.J. Xu, M.L. Daley, Design of optimal digital filter using a parallel genetic algorithm, *Circuits and Systems II: Analog and Digital Signal Processing, IEEE Transactions On* 42 (1995) 673–675.
- [29] C.-L. Chen, A.N. Willson Jr, A trellis search algorithm for the design of FIR filters with signed-powers-of-two coefficients, *Circuits and Systems II: Analog and Digital Signal Processing, IEEE Transactions On* 46 (1999) 29–39.
- [30] D. Kodek, K. Steiglitz, Comparison of optimal and local search methods for designing finite wordlength FIR digital filters, *Circuits and Systems, IEEE Transactions On* 28 (1981) 28–32.